

AccoTEST 自动化接口编程手册

(Rev 1.00)

目 录

目 录.....	2
1 使用说明.....	6
2 快速应用指导.....	7
2.1 应用模式一	7
2.2 应用模式二	8
3 CONTROL 相关接口函数.....	11
3.1 RunControlNoReg()	11
3.2 SwitchUser()	11
3.3 CloseControl()	12
3.4 SetControlRemoteMode()	13
3.5 GetStationID()	13
3.6 SetCurrentStation()	14
3.7 GetUserPriority()	14
3.8 GetCheckResult()	15
3.9 StartCheck()	15
3.10 CloseCheck()	16
3.11 GetHardwareStatus()	16
4 TESTUI 启动与测试基本设置接口函数.....	18
4.1 RunTestui()	18
4.2 RunTestuiWithStationid()	18
4.3 RunTestuiWithFilename()	19
4.4 CreateNewLotFile_Acco()	20
4.5 AppendStdLotMsg3()	20
4.6 SetPgsFileName()	24
4.7 LoadFile()	24
4.8 AutoLoadPgsFromZipFile()	25
4.9 UpdateLotInfo()	26
4.10 SetSaveFileName()	27
4.11 SetDatalogFileName()	28
4.12 SetLocalPath()	29
4.13 SetDatalogPath()	29
4.14 SetSumPath()	30
4.15 SetSummaryLogSaveOption()	31
4.16 AutoExportSet()	31
4.17 SetTesterID()	32
4.18 UpdateQAParamInfo()	33

5	测试控制接口函数.....	35
5.1	StartTest()	35
5.2	StartTestWithstationid()	35
5.3	SingleTest()	36
5.4	StopTest()	36
5.5	CloseTestui()	37
5.6	ForceCloseTestui()	37
5.7	EnableShellClose()	38
5.8	EnableUserCommand()	38
5.9	EnableQATest()	38
5.10	SetAutoLoadMode()	39
5.11	SetInterFaceMode()	40
5.12	DisAutoShowLotInfo()	40
5.13	FileUnLoadWithoutConfirm()	41
5.14	NewLot()	41
5.15	EndCurrentLog()	42
5.16	DisableNewLotFunction()	42
5.17	DisableUnloadFunction()	43
5.18	DisableloadFunction()	43
5.19	DisableEditFunction()	43
5.20	DisableTestFunctionStatus()	44
5.21	SetTestuiRemoteMode()	44
6	测试状态与信息获取.....	46
6.1	IsTestFileLoad()	46
6.2	IsDataSave()	46
6.3	GetCurPgsName()	47
6.4	GetDataLogFileName()	48
6.5	GetExportFilename()	48
6.6	GetInfo()	49
6.7	GetInfoWithSTDFID()	50
6.8	CurTestUIExist()	51
6.9	GetTestSiteCount()	51
6.10	GetTestedResult()	52
6.11	GetTestuiTestingStatus()	52
6.12	GetExportResult()	53
6.13	GetExportResultWithOuttime()	54
6.14	GetExportResultWithMdbVerify()	54
7	CONTROL 高级函数接口.....	56
7.1	StartCalibration()	56
7.2	GetCalStatus()	56
7.3	CloseCalibration()	57
8	测试信息设置高级函数接口.....	58

8.1	RunTestuiWithAllOption()	58
8.2	RunTestuiWithAllOption2()	59
8.3	EnableUpdateLotIDfromProbe()	60
8.4	SetLDFDelAttr()	61
8.5	SetStdExt()	61
8.6	SetParamTextColor()	62
8.7	SetTestProgramSourceInfo()	63
8.8	SummaryExportTxtMode()	63
8.9	SummaryExportSumMode()	64
8.10	SetWholeStdRecord()	64
9	测试控制高级函数接口	70
9.1	SetSiteStatus()	70
9.2	UpdateSystemEvent()	70
9.3	SetMaxAutoRunCount()	71
9.4	EnableSBL()	71
9.5	OnlyEnableSBL()	72
9.6	SetSBLimit()	73
9.7	SetSBLYieldDiffLimit()	73
9.8	SetSBLAffectedSite()	74
9.9	SetRecentYield()	75
9.10	SetYieldDifferent()	76
9.11	SetCorrelationMode()	76
9.12	LaunchAutoCorrelation()	77
9.13	EnableSPL()	78
9.14	SetSPLApplySampleMode()	79
9.15	SetSPLYieldSampleCount()	79
9.16	SetSPLYieldDIFFSampleCount()	80
9.17	SetSPLAverageDiffSampleCount()	80
9.18	SetSPLAverageRangeSampleCount()	81
9.19	SetSPLFluctuationRangeSampleCount()	81
9.20	SetSPLStackingDetectionSampleCount()	82
9.21	SetSPLYieldLimit()	82
9.22	SetSPLYieldDiffLimit()	83
9.23	SetSPLAverageDiffLimit()	84
9.24	SetSPLAverageRangeLimit()	85
9.25	SetSPLFluctuationRangeLimit()	86
9.26	SetSPLStackingDetectionLimit()	87
9.27	ClearSPLSet()	87
9.28	SetContinuallyFailStop()	88
9.29	SetContinuallyPassStop()	89
10	测试信息获取高级函数接口	90
10.1	GetSiteStatus()	90
10.2	GetSubUnitCount()	90

10.3	GetTestValue()	91
10.4	GetParamCount()	92
10.5	GetSubUnitCountWithParamIndex()	92
10.6	GetSiteTestResult()	93
10.7	GetParamTestResult()	94
10.8	GetBinCount()	95
10.9	GetBinInfo()	95
11	其它接口函数	97
11.1	CompactFile()	97

1 使用说明

以下接口函数主要用于提供用户实现自动化测试的标准函数，供用户的自动化控制程序调用，从而满足用户的自动化测试需求，特别是满足 OP 实现自动化测试。所有的标准接口函数统一封装集成在一个通用、统一的 `Sts8200Interface.dll` 文件中。用户只需要调用这些标准接口函数，去实现用户期望的功能。至于这些标准接口函数的功能如何实现，则由此 DLL 文件负责实现。

以下函数的运用都是以系统处于正常状态，测试程序也已经调试并验证通过。如果运用过程中，ACCO 的测试系统弹出各种提示与警告信息，请先处理并排除各种提示与警告信息，以保证后续接口函数能够正常实现其功能。

本接口函数主要用于控制 ACCO 测试系统，实现自动化测试的功能。其它功能如访问 ERP 系统或 EMS 系统、测试程序下载、数据文件上传等功能由用户的自动化控制程序自行实现。

如果用户是双 Station 乒乓测试，则自动化控制函数如果不包含 Station ID 参数，必须先调用 `SetCurrentStation` 函数切换 Station，然后再调用相关接口函数，从而实现对每个 Station 单独进行控制。

本文档中的接口函数都是 `stdcall` 调用

2 快速应用指导

下面列出两种推荐的自动化接口函数应用方式。本节主要说明各函数的调用顺序，具体各函数的调用返回值请在实际应用时自行添加处理。

2.1 应用模式一

本模式主要包括下面功能：以 OP 权限启动 Control 并登录、启动 TestUI，设置 LOT 信息项、设置不自动弹出 LOT 信息框，载入测试程序、直接设置 DataLog 文件名，设置转换输出，启动测试。最后停止测试，关闭 TestUI。本模式由于使用了 SetInterFaceMode 函数，用户可以在启动测试前，进行一些设置后，然后再通过 StartTest 函数启动测试。

```
//启动 Control 程序，必须提前创建好账户 oper
RunControlNoReg("oper", "oper");
Sleep(2000);                                // 延时 2 秒

//启动 station A
RunTestuiWithStationid(0);
Sleep(1000);                                // 延时 1 秒

//生成新的 lot 信息文件。
CreateNewLotFile_Acco();
//添加 lot 信息项
AppendStdflotMsg3("LotID",0,"Pmclint5467", 1, 0);
AppendStdflotMsg3("Device ID", 4, "OM586", 1, 0);
AppendStdflotMsg3("Test Code", 10, "FT", 1, 0);

//设置自动测试模式，OP 权限下，载入按钮无效
SetAutoLoadMode(1);
SetInterFaceMode(1);

//禁止弹出 LOT 信息框
DisAutoShowLotInfo();

//设置 Pgs 文件名。
```

```

SetPgsFileName("C:\\AccoTest2021\\AccoTEST
System\\TESTFILE\\ACSM_Example\\ACSM_Example.pgs");

//调入测试程序。
LoadFile(10000);

// 设置转换输出 Excel
AutoExportSet(1);

//设置存储文件名（包括存储路径）
SetSaveFileName("C:\\AccoTest2021\\AccoTEST
System\\DATALOG\\Lot15458Wafer9875624_2021-11-10_1940_40");

//设置输出 CSV 格式的 Summary 文件
SetSummaryLogSaveOption(1,1,1);

    StartTest();    // 启动测试

// 测试结束后，调下面函数结束测试
// 停止测试
StopTest();

// 关闭 Testui
ForceCloseTestui();

```

2.2 应用模式二

本模式主要包括下面功能：以 OP 权限启动 Control 并登录、启动 TestUI，设置 LOT 信息项，设置 DataLog 文件名规则，设置不自动弹出 LOT 信息框，载入测试程序，设置转换输出，启动测试。最后停止测试、关闭 TestUI。本模式与前面模式的区别是使用了设置 DataLog 文件名规则。

```

//启动 Control 程序，必须提前创建好账户 oper
RunControlNoReg("oper", "oper");
Sleep(2000);                                // 延时 2 秒

//启动 station A
RunTestuiWithStationid(0);
Sleep(1000);                                // 延时 1 秒

//生成新的 lot 信息文件。
CreateNewLotFile_Acco();

```

```

//添加 lot 信息项
AppendStdflotMsg3("Test facility ID",17,"VTEST",1,0);
AppendStdflotMsg3("Test flow ID",23,"CP",1,0);
AppendStdflotMsg3("Product family ID",15,"PAC",1,0);
AppendStdflotMsg3("Part type",1,"TPL910Q2_Q",1,0);
AppendStdflotMsg3("Lot Id",0,"Q6U884.1",1,1);
AppendStdflotMsg3("Name of node that generated data",2,"TTA-09",1,0);

AppendStdflotMsg3("(Sdr)Handler or prober ID",52,"POS-111",1,0);
AppendStdflotMsg3("(Sdr)Probe card ID",54,"TPL910A1-PC-01",1,0);
AppendStdflotMsg3("WAFER_ID",36,"WAFER_ID",1,1);
AppendStdflotMsg3("Test phase or step code",10,"CP2",1,0);

AppendStdflotMsg3("Lot retest code",31,"RP0",1,0);    //}
AppendStdflotMsg3("Operator name or ID",7,"2223",1,0);

//设置 DataLog 文件名规则
SetDatalogFileName( "[Test facility ID]_[Test flow ID]_[Product family
ID]_[Part                                     type]_[Lot
Id]_[WAFER_ID]_[DATE(yyyymmdd)][Time(hhmm_ss)]");

    //设置自动测试模式，OP 权限下，载入按钮无效
SetAutoLoadMode(1);
SetInterFaceMode(1);

//禁止弹出 LOT 信息框
DisAutoShowLotInfo();

//设置 Pgs 文件名。
SetPgsFileName("C:\\AccoTest2021\\AccoTEST
System\\TESTFILE\\ACSM_Example\\ACSM_Example.pgs");

//调入测试程序。
LoadFile(10000);

//设置输出 CSV 格式的 Summary 文件
SetSummaryLogSaveOption(1,1,1);

// 设置转换输出 Excel、CSV、STDF
AutoExportSet(7);

StartTest();    // 启动测试

// 测试结束后，调下面函数结束测试
// 停止测试
StopTest();

```

```
// 关闭 Testui  
ForceCloseTestui();
```

3 Control 相关接口函数

3.1 RunControlNoReg()

```
int RunControlNoReg(char* username; char* password);
```

Parameters

Username

用户名

Password

密码

Return Values

0 : 成功

1: 在此命令之前 Control 已被启动

-1: 没有启动 Control, 获取 Control 句柄为 0

-9: Control 虽然被启动, 但没有成功登录, 可能是用户名与密码错误

Remarks

启动 Control 程序

此函数必须在下面所有函数之前启动。如果 Control 已启动则不用再启动。

一般不建议用户外壳控制程序使用此函数来启动 Control 程序。而是通过 ACCO 的测试系统来启动 Control 程序。

Example

```
RunControlNoReg("oper", "oper");
```

3.2 SwitchUser()

```
int SwitchUser(char* UserName; char* Password);
```

Parameters

UserName

用户名

Password

密码

Return Values

- 0: 成功
- 1: Control 没有启动
- 2: 无法切换用户,有程序没有关闭
- 7: 无法切换用户,开机上电超时中
- 9: 没有成功切换用户, 可能是用户名与密码错误

Remarks

切换 Control 用户

Example

```
SwitchUser("oper", "oper");
```

3.3 CloseControl()

```
int CloseControl();
```

Return Values

- 0 : 完全关闭
- >0 : 硬件正在运行中, 不能关闭
- 1 : 没有成功地关闭 Control 。原因有可能是 Control 没有启动不需要关闭 Control 或其它原因
- 2 : 连续测试中, 没有成功地关闭 Control

Remarks

关闭 Control

Example

```
CloseControl();
```

3.4 SetControlRemoteMode()

```
int SetControlRemoteMode();
```

Return Values

- 0 : 成功
- 1 : Control 没有启动

Remarks

设置 Control 为自动化 UI 控制模式。不允许从 Control 界面启动 testui 进行测试、不允许切换用户、不允许通过 Control 启动 VC Project 进行编辑功能

Example

```
SetControlRemoteMode();
```

3.5 GetStationID()

```
int GetStationID();
```

Return Values

- 0 : STATION A 可启动
- 1 : STATION B 可启动
- 254 : Control 没启动
- 255 : 没有可用于启动的 Station 的 ID 号

Remarks

双 Station 系统，得到可用于启动的 Station 的 ID 号。

注意：对于系统为单 Station 的系统，此函数不需要调用。

Example

```
int curstationId = GetStationID();
```

3.6 SetCurrentStation()

```
int SetCurrentStation(int stationid);
```

Parameters

Stationed

0 : 代表 STATION A 1 : 代表 STATION B

Return Values

-1 : Control 没有启动

-9 : Control 已启动，但没有登录

-2 : 当前 station 的 testui 没有启动

0 : 当前 station 的 testui 已启动，且载入了测试程序

1 : 当前 station 的 testui 已启动，但没有载入测试程序

Remarks

切换当前 station，并判断状态。返回结果为非负值时，后续所有不带 stationid 的函数都是针对此 station 的 testui。

Example

```
SetCurrentStation(0);
```

3.7 GetUserPriority()

```
int GetUserPriority();
```

Return Values

1 : OP

2 : ENGINEER

3 : ADMIN

0 : 用户没有登陆或 Control 没有启动

Remarks

得到用户权限

Example

```
int curUserPriority = GetUserPriority();
```

3.8 GetCheckResult()

```
int GetCheckResult();
```

Return Values

- 255 : 没有启动 Control
- 0 : 自检合格
- 1 : 自检失效

Remarks

得到自检结果状态。Control 启动后没有进行过自检，自检结果状态为失效状态

Example

```
int curCheckResult = GetCheckResult();
```

3.9 StartCheck()

```
int StartCheck();
```

Return Values

- 0 : 启动自检
- 1 : 没有启动 Control
- 2 : 没有上电，无法自检

Remarks

启动自检

Example

```
StartCheck();
```

3.10 CloseCheck()

```
int CloseCheck();
```

Return Values

- 0 : 成功关闭自检
- 1 : 没有启动 Control
- 2 : 自检中, 无法关闭自检
- 3 : 无法关闭自检, 自检窗口为 8107 系统版本

Remarks

关闭自检窗口

Example

```
CloseCheck();
```

3.11 GetHardwareStatus()

```
int GetHardwareStatus();
```

Return Values

- 255 : Control 没启动
- 9 : 断电状态
- 1 : 硬件没准备好(开机初始化)
- 0 : 硬件空闲
- 1 : 硬件被 TESTUI 使用
- 2 : 硬件被 STSCAL 使用
- 3 : 硬件被 CHECK 使用
- 4 : 硬件被 Diagnosis 使用

Remarks

得到硬件状态

Example

```
int curHardwareStatus= GetHardwareStatus();
```

4 TESTUI 启动与测试基本设置接口函数

4.1 RunTestui()

```
int RunTestui();
```

Return Values

- 0 : 启动了 STATION A 对应的 Testui
- 1 : 启动了 STATION B 对应的 Testui
- 255 : Control 没有启动
- 254 : no more station
- 253 : Testui 没有启动成功

Remarks

启动 Testui 程序。（testui 按 A、B 顺序启动）

Example

```
RunTestui();
```

4.2 RunTestuiWithStationid()

```
int RunTestuiWithStationid(int stationid);
```

Parameters

Stationed

- 0 : 代表 STATION A 1 : 代表 STATION B

Return Values

- 1 : Control 没有启动
- 2 : 当前 station 的 testui 已启动
- 0 : 命令不成功
- >0: 命令成功，返回 testUI 的句柄号

Remarks

启动 testui（通过 stationid，在不启动 station A 的情况下直接启动 Station B

Example

```
RunTestuiWithStationid(0);
```

4.3 RunTestuiWithFilename()

```
int RunTestuiWithFilename(char* pgsfilename; int delaytime);
```

Parameters

Pgsfilename

测试程序 PGS 文件名

Delaytime

调入程序函数超时退出时间（ms）

Return Values

0 : 启动了 STATION A 对应的 Testui 并调入测试程序

1 : 启动了 STATION B 对应的 Testui 并调入测试程序

254 : 调入程序超时退出

255 : Testui 没有启动成功

Remarks

直接启动 Testui 程序并调入测试程序（testui 按 A、B 顺序启动）

注意：此函数由下面的 RunTestui、SetPgsFileName、LoadFile 三个函数组合而成。

Example

```
char    curpgsname[]    =    "    C:\\AccoTest2021\\AccoTEST  
System\\TESTFILE\\ACM_Example\\ ACM_Example.pgs";  
RunTestuiWithFilename(curpgsname,10000);
```

4.4 CreateNewLotFile_Acco()

```
int CreateNewLotFile_Acco();
```

Return Values

0 : 成功创建 lot 信息文件

Remarks

生成新的 lot 信息文件。

每启动一个 Station，如果要添加不同的 lot 信息项，则必须调用此函数重新创建 lot 信息文件。在双 Station 时尤其要注意调用此函数。

此函数必须在 LoadFile 函数前先调用。如果在测试程序已调入的情况下调用此命令，虽然能更改 lot 信息文件，但不会影响 TestUI 已载入测试程序的 lot 信息项。

Example

```
CreateNewLotFile_Acco();
```

4.5 AppendStdLotMsg3()

```
int AppendStdLotMsg3(char* LotTitle;int stdfId; char* LotValue;  
unsigned short curView; unsigned short curUpdate);
```

Parameters

LotTitle

lot 信息项标题

stdfId

对应 stdf 存储的标识号，下面是 stdf 项标识表

LotValue

lot 信息项内容值

curView

是否在 LOT 信息输入窗口显示，0：不显示 其它：显示

curUpdate

是否用 Probe 获取的值更新，0：不更新 其它值：更新

Return Values

-1 : LotTitle 为空

-2 : LotValue 为空

0 : 成功创建 lot 信息文件

1 : 测试程序已调入，不能更改 lot 信息文件

Remarks

增加新的 lot 信息（属于 StdF 的标准信息）

此函数必须在 LoadFile 函数前先调用

stdf 存储标识号	StdF 简称	stdf 对应项意义
0	(Mir)LOT_ID	Lot Id
1	(Mir)PART_TYP	Part type
2	(Mir)NODE_NAM	Name of node that generated data
3	(Mir)TSTR_TYP	Tester type
4	(Mir)JOB_NAM	Job name
5	(Mir)JOB_REV	Job revision number
6	(Mir)SBLot_ID	Sublot ID
7	(Mir)OPER_NAM	Operator name or ID
8	(Mir)EXEC_TYP	Tester executive software type
9	(Mir)EXEC_VER	Tester exec software version number
10	(Mir)TEST_COD	Test phase or step code
11	(Mir)TST_TEMP	Test temperature
12	(Mir)USER_TXT	Generic user text
13	(Mir)AUX_FILE	Name of auxiliary data file
14	(Mir)PKG_TYP	Package type
15	(Mir)FAMILY_ID	Product family ID
16	(Mir)DATE_COD	Date code

17	(Mir)FACIL_ID	Test facility ID
18	(Mir)FLOOR_ID	Test floor ID
19	(Mir)PROC_ID	Fabrication process ID
20	(Mir)OPER_FRQ	Operation frequency or step
21	(Mir)SPEC_NAM	Test specification name
22	(Mir)SPEC_VER	Test specification version number
23	(Mir)FLOW_ID	Test flow ID
24	(Mir)SETUP_ID	Test setup ID
25	(Mir)DSGN_REV	Device design revision
26	(Mir)ENG_ID	Engineering lot ID
27	(Mir)ROM_COD	ROM code ID
28	(Mir)SERL_NUM	Tester serial number
29	(Mir)SUPR_NAM	Supervisor name or ID
30	(Mir)MODE_COD	Test mode code
31	(Mir)RTST_COD	Lot retest code
32	(Mir)PROT_COD	Data protection code
33	(Mir)BURN_TIM	Burn-in time
34	(Mir)CMOD_COD	Command mode code
35	Customer define	Customer define
36	(Wrr)WAFER_ID	(Wrr)Wafer_ID
37	(Wrr)FABWF_ID	(Wrr)FabWF_ID
38	(Wrr)FRAME_ID	(Wrr)Frame_ID
39	(Wrr)MASK_ID	(Wrr)Mask_ID
40	(Wrr)USR_DESC	(Wrr)User_Desc
41	(Wrr)EXC_DESC	(Wrr)Exc_Desc
42	(Wcr)WAFR_SIZ	(Wcr)Diameter of wafer in WF_UNITS
43	(Wcr)DIE_HT	(Wcr)Height of die in WF_UNITS
44	(Wcr)DIE_WID	(Wcr)Width of die in WF_UNITS
45	(Wcr)WF_UNITS	(Wcr)Units for wafer and die dimensions
46	(Wcr)WF_FLAT	(Wcr)Orientation of wafer flat

47	(Wcr)CENTER_X	(Wcr)X coordinate of center die on wafer
48	(Wcr)CENTER_Y	(Wcr)Y coordinate of center die on wafer
49	(Wcr)POS_X	(Wcr)Positive X direction of wafer
50	(Wcr)POS_Y	(Wcr)Positive Y direction of wafer
51	(Sdr)HAND_TYP	(Sdr)Handler or prober type
52	(Sdr)HAND_ID	(Sdr)Handler or prober ID
53	(Sdr)CARD_TYP	(Sdr)Probe card type
54	(Sdr)CARD_ID	(Sdr)Probe card ID
55	(Sdr)LOAD_TYP	(Sdr)Load board type
56	(Sdr)LOAD_ID	(Sdr)Load board ID
57	(Sdr)DIB_TYP	(Sdr)DIB board type
58	(Sdr)DIB_ID	(Sdr)DIB board ID
59	(Sdr)CABL_TYP	(Sdr)Interface cable type
60	(Sdr)CABL_ID	(Sdr)Interface cable ID
61	(Sdr)CONT_TYP	(Sdr)Handler contactor type
62	(Sdr)CONT_ID	(Sdr)Handler contactor ID
63	(Sdr)LASR_TYP	(Sdr)Laser type
64	(Sdr)Laser ID	(Sdr)Laser ID
65	(Sdr)EXTR_TYP	(Sdr)Extra equipment type field
66	(Sdr)EXTR_ID	(Sdr)Extra equipment ID
67	(Wir)WAFER_ID	(Wir)WAFER_ID

stdfld 为 0 至 34 是对应 stdf 记录结构 Mir 的信息项。

stdfld 为 36 至 41 是对应 stdf 记录结构 Wrr 的信息项。

stdfld 为 42 至 50 是对应 stdf 记录结构 Wcr 的信息项。

stdfld 为 51 至 66 是对应 stdf 记录结构 Sdr 的信息项。

stdfld 为 67 是对应 stdf 记录结构 Wir 的信息项。

stdfld 为 35 的信息存储专用于设置无法对应 STDF 定义项的信息，可以设

置多个 stdfId 为 35 的信息项。

Example

```
AppendStdflotMsg3("Operator name", 7, "oper", 1, 0);
```

4.6 SetPgsFileName()

```
int SetPgsFileName(char* PgsFileName);
```

Parameters

pgsfilename

测试程序 PGS 文件名，包括路径

Return Values

- 0 : 成功传入测试程序文件名
- 1 : Testui 没有启动
- 2 : 设置的测试程序文件不正确或文件不存在

Remarks

设置 Pgs 文件名。此函数必须在 LoadFile 函数前先调用。

Example

```
SetPgsFileName("C:\\AccoTest2021\\AccoTEST  
System\\TESTFILE\\ACSM_Example\\ACSM_Example.pgs");
```

4.7 LoadFile()

```
int LoadFile(int delaytime);
```

Parameters

Delaytime

调入程序函数超时退出时间 (ms)

Return Values

- 0 : 成功调入测试程序
- 254 : 调入程序超时退出

Remarks

调入测试程序。此函数必须在 SetPgsFileName 函数之后调用。

另外，如果前面用 AppendStdflotMsg3 函数添加了 Lot 信息项，则此函数自动调用 SetAutoLoadMode 函数设置自动属性。AutoLoad 属性会在载入测试程序时使用 AppendStdflotMsg3 函数添加的 Lot 信息项；在 op 模式下，调用程序后 AutoLoad 属性会自动弹出 lot 信息对话框，点击 ok 后，程序开始自动测试。

Example

```
LoadFile(10000);
```

4.8 AutoLoadPgsFromZipFile()

```
int AutoLoadPgsFromZipFile(char* ZipFileName; char* PgsFileName; int delaytime);
```

Parameters

ZipFileName

压缩包文件名，如果不包含路径或者虽然包含路径，但对应的 Zip 文件不存在，则在 testui 设置的载入程序路径中查找相应的文件

PgsFileName

要载入的测试程序文件名，不包括路径，如果没有后缀会自动添加 PGS 后缀。

delaytime

调入程序函数超时退出时间（ms）

Return Values

- 0 : 成功调入测试程序
- 1 : testui 没有启动
- 1: 没有找到 ZipFileName 文件

- 2: 找到多个 ZipFileName 文件
- 3: 没有找到 PgsFileName 文件
- 4: 找到多个 PgsFileName 文件
- 254 : 调入程序超时退出

Remarks

从远程服务器自动下载测试程序 ZIP 包到本地，自动解压，自动载入测试程序

Example

```
char ZipFileName[] = "Z:\\ast2100\\A.zip";
AutoLoadPgsFromZipFile(ZipFileName, " A.pgs", 20000);
```

4.9 UpdateLotInfo()

```
int UpdateLotInfo(int stationid;char* LotTitle;int stdfId; char* LotValue);
```

Parameters

Stationed

0 代表 STATION A 1 代表 STATION B

LotTitle

lot 信息名称

stdfId

lot 信息项对应的 stdf 号

LotTitle

lot 信息内容

Return Values

- 1 : Control 没有启动
- 2 : 当前 station 的 testui 没有启动
- 3 : 当前 station 的 testui 已启动,但没有调入测试程序,未知原因
- 4 : testui 已调程序的 lot 信息项数为 0

-5 : 没有匹配到已调程序的 lot 信息项（根据 LotTitle、stdfld 两项内容匹配，大小写敏感）

0 : 更新成功

Remarks

更新已载入测试程序的 lot 信息

此函数必须在 LoadFile 函数后才能调用。

Example

```
UpdateLotInfo(0, "Lot ID", 0, "AS368");
```

4.10 SetSaveFileName()

```
int SetSaveFileName(char* SaveFileName);
```

Parameters

SaveFileName

数据存储文件名

Return Values

0 : 成功传入数据存储文件名

1 : 测试程序没有调入

-1 : 存储的数据文件没有创建成功

-2 : 新老文件名相同

Remarks

此函数必须在 LoadFile 函数之后、启动测试之前调用，则可以设置存储文件名（包括存储路径），并创建数据文件；如果在开始测试并保存数据文件之后，调用本接口传过来的数据文件名，如果与原数据文件名不一致，则在测试结束后更改存储文件名为最新的数据文件名；如果在开始测试并没有保存数据文件的情况下调用本函数，之后，则可以设置存储文件名（包括存储路径），并创建数据文件，开始进行存储。

Example

SetSaveFileName("C:\\AccoTest2021\\AccoTEST 458Wafer9875624_2021 - 11 - 10_1940_40");	System\\DATALOG\\Lot	15
--	----------------------	----

4.11 SetDatalogFileName()

```
int SetDatalogFileName(char* DatalogFileName);
```

Parameters

DatalogFileName

数据存储文件名命名规则

下面是一 Datalog 命名规则示例。通过示例说明 Datalog 命名规则组成。

[Subcon]_[Tester ID]_[Process]_[Device Name]_[PGSFileName]_[FAB Lot
ID]_[Subcon LOTID]_[WAFER ID]_[Test insertion name]_[Date(yyymmdd)][Time]

- 所有文件项必须在[]括号之中
- 两套[]之间的字符为连接符，如果两套[]之间没有字符，说明此两项之间没有连接符
- 下面字符串作为文件项为固定意义的字符串
 - PGSFileName: 测试程序名（不包括后缀）
 - Date(yyymmdd): 日期，格式为 yyymmdd
 - Date(mmddyyyy): 日期，格式为 mmddyyyy
 - Time: 时间，格式为 hhmmss
 - Date(xxxxxxxx): 日期，自定义格式为 xxxxxxxx
 - Time(xxxxxx): 时间，自定义格式为 xxxxxx
- 其它文件名必须与 lot 信息项的项名完全相同

Return Values

- 0 : 成功传入数据存储文件名命名规则
- 1 : DatalogFileName 为空或不符合命名规则，无法设置
- 2 : DatalogFileName 无法找到对应项，无法设置

Remarks

设置测试数据存储 Datalog 命名规则。在启动测试后，则系统自动按照命名规则产生存储文件名（不包括存储路径），并创建数据文件，开始进行存储。

此函数必须在通过接口函数如 AppendLotMsg3 完成所有 lot 信息添加之后、

在 LoadFile 函数之前调用

Example

```
char DatalogFileName[] = "[Lot Id]_[WAFER ID]_[Date(yyyymmdd)][Time(hhmmss)]";  
SetDatalogFileName(curDatalogFileName);
```

4.12 SetLocalPath()

```
int SetLocalPath(int LocalValue; char* Localpath);
```

Parameters

LocalValue:

- 0: 不设置本地存储路径
- 1: 设置本地存储路径,用缺省数据存储路径作为本地存储路径,即使用安装目录下面的 datalog 子目录。
- 2: 设置本地存储路径,用 Localpath 作为本地存储路径

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动
- 1: 模式 2 时,设置的本地路径为空

Remarks

设置本地存储路径

此函数必须在 LoadFile 函数之后、SetSaveFileName 函数之前调用

Example

```
SetLocalPath("C:\\accotest\\STS8200CROSS\\DATALOG\\");
```

4.13 SetDatalogPath()

```
int SetDatalogPath(char* DefDatalogPath);
```

Parameters

DefDatalogPath

Datalog 存储路径

Return Values

- 1 : 路径为空
- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

设置 Datalog 存储路径

此函数必须在 LoadFile 函数之后、SetSaveFileName 函数之前调用

Example

```
SetDatalogPath("C:\\accotest\\STS8200CROSS\\DATALOG\\");
```

4.14 SetSumPath()

```
int SetSumPath(char* Sumpath);
```

Parameters

Sumpath

Sum 存储路径

Return Values

- 1 : 路径为空
- 0 : 成功传入 Sum 存储路径
- 1 : Testui 没有启动

Remarks

设置 Sum 存储路径

Example

```
SetSumPath("C:\\accotest\\STS8200CROSS\\DATALOG\\");
```

4.15 SetSummaryLogSaveOption()

```
int SetSummaryLogSaveOption(int EnableFlag; unsigned short  
similarDatalogFilename; unsigned short sameDatalogPath);
```

Parameters

EnableFlag

- 0 -- 禁止自动输出 Summary 文件
- 1 -- 自动输出 summary 文件

similarDatalogFilename

- (summary 输出文件名与 datalog 文件名相似)
- 0: 不相似
 - 1: 相似

similarDatalogPath

- (summary 输出路径与 datalog 路径相同)
- 0: 相同路径设置不被选择
 - 1: 相同路径设置被选择

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

设置 Sum 存储相关设置（此函数需更新 2021.02.08 后的版本）

Example

```
SetSummaryLogSaveOption(1,1,1);
```

4.16 AutoExportSet()

```
int AutoExportSet(int AutoExportValue);
```

Parameters

AutoExportValue

自动输出各种类型的数据文件的设置值

Bit 0 为 1: 输出 Excel 文件

Bit 1 为 1: 输出 Csv 文件

Bit 2 为 1: 输出 STDF 文件

Bit 3 为 1: 在 Bit 4 为 0 的情况下, 输出紧凑格式的 Text 文件

Bit 4 为 1: 在 Bit 3 为 1 的情况下, 输出非紧凑格式的 Text 文件

Bit 5 为 1: 输出用户自定义文件 (必须有用户自定义输出 DLL 支持)

Return Values

0 : 命令成功

1 : Testui 没有启动

Remarks

自动输出各种类型的数据文件的设置

此函数必须在创建数据库文件之前调用, 因为涉及实时 STDF 输出与创建数据库同步问题

Example

```
AutoExportSet(7); // 输出 Excel、CSV、STDF
```

4.17 SetTesterID()

```
int SetTesterID(char* TesterID);
```

Parameters

TesterID

测试设备仪器编号

Return Values

0 : 成功传入测试设备仪器编号

1 : Testui 没有启动

Remarks

设置测试设备仪器编号

Example

```
SetTesterID("20213456");
```

4.18 UpdateQAParamInfo()

```
int UpdateQAParamInfo(int stationid; unsigned short curQAEEnable;  
unsigned short ShowQAParamDlg; unsigned short curQAType; int  
curQASetNumber;int curMaxQANumber;int curLotNumber; double  
curDefaultPassRate);
```

Parameters

stationid

0: 代表 STATION A 1: 代表 STATION B

curQAEEnable

是否允许 QA 1: 代表允许 0: 不允许

ShowQAParamDlg

是否显示 QA 设置参数输入框 1 代表允许 0: 不允许

curQAType

QA 类型 1: 根据良率做 QA; 2: 直接设置 QA 间隔数

curQASetNumber

QA 间隔数, 此参数在 curQAType 为 2 时有意义, 在 curQAEEnable 有效时, 此值必须大于 0;

curMaxQANumber

最大 QA 数

curLotNumber

批次数, 此参数在 curQAType 为 1 时有意义

curDefaultPassRate

良率, 此参数在 curQAType 为 1 时有意义

Return Values

- 1 : Control 没有启动
- 2 : 当前 station 的 testui 没有启动
- 0 : 命令执行

Remarks

更新 QA 测试相关设置信息

此函数必须在 LoadFile 函数后，SetSaveFileName 函数及启动测试函数前调用

Example

```
UpdateQAParamInfo(0,1,0,2,5,1000,3000,0.7);
```

5 测试控制接口函数

5.1 StartTest()

```
int StartTest();
```

Return Values

- 0 : 命令成功
- 1 : 测试程序没有调入

Remarks

启动测试

自动测试模式，op 用户调入程序后，会自动启动测试。所以 op 用户可不调用此函数。

Example

```
StartTest();
```

5.2 StartTestWithstationid()

```
Int StartTestWithstationid(int stationid);
```

Parameters

stationid

- 0 : 代表 STATION A
- 1 : 代表 STATION B

Return Values

- 1 : Control 没有启动
- 2 : 当前 station 的 testui 没有启动
- 3 : 当前 station 的 testui 没有调入程序
- 0 : 命令成功

Remarks

当前 Station ID 号的 TestUI 启动测试

自动测试模式，op 用户调入程序后，如果会自动启动测试，则 op 用户可不调用此函数

Example

```
int res= StartTestWithstationid(0);
```

5.3 SingleTest()

```
int SingleTest();
```

Return Values

- 0 : 命令成功
- 1 : 测试程序没有调入
- 2 : 已启动测试。重复命令。

Remarks

启动测试，单测

Example

```
SingleTest();
```

5.4 StopTest()

```
int StopTest();
```

Return Values

- 0 : 命令成功
- 1 : 测试程序没有调入

Remarks

停止测试

Example

```
StopTest();
```

5.5 CloseTestui()

```
int CloseTestui();
```

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

关闭 Testui 程序。如果 Testui 正测试，会有提示。

Example

```
CloseTestui();
```

5.6 ForceCloseTestui()

```
int ForceCloseTestui();
```

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

停止测试，关闭 Testui 程序。设置了 EnableShellClose 的窗口必须通过本函数才能关闭。

Example

```
ForceCloseTestui();
```

5.7 EnableShellClose()

```
int EnableShellClose(unsigned short OnlyShellClose);
```

Parameters

OnlyShellClose

Testui 程序关闭属性（如果为 1，则 Testui 必须由用户自动化程序调关闭函数 ForceCloseTestui 关闭 Testui）

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

设置 TESTUI 关闭属性

Example

```
EnableShellClose(1);
```

5.8 EnableUserCommand()

```
int EnableUserCommand();
```

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

开启用户自定义插件命令（机台状况监控功能的开关在此）

Example

```
EnableUserCommand();
```

5.9 EnableQATest()

```
int EnableQATest();
```

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

允许在线 QA 测试

Example

```
EnableQATest();
```

5.10 SetAutoLoadMode()

```
int SetAutoLoadMode(unsigned short AutoMode);
```

Parameters

AutoMode

自动测试模式属性 1: 置为自动测试模式

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

设置 TESTUI 是否为自动测试模式属性。此函数必须在 LoadFile 函数前先调用。如果是自动测试模式，有下面三特征：

- A、如果选择了输出 CSV 格式的 Summary 文件，Summary 文件名与数据文件名相似，后缀 “.mdb”变为 “_sum.csv”;
- B、OP 权限下，载入按钮无效
- C、自动开始测试。

Example

```
SetAutoLoadMode(1);
```

5.11 SetInterFaceMode()

```
int SetInterFaceMode(unsigned short InterFaceMode);
```

Parameters

InterFaceMode

接口方式

此变量作为 AutoLoadMode 的补充, 将 AutoLoadMode 模式的一些功能开放出来

0 : 不开放 (AutoLoadMode 模式下, op 模式调入程序后, 会自动运行, 载入按钮无效。summary 文件名与数据文件名相似, 后缀变为 “_sum.csv”)

1 : op 模式调入程序后, 不会自动运行。summary 文件存储按界面设置进行存储

Return Values

0 : 命令成功

1 : Testui 没有启动

Remarks

设置 TESTUI 为自动测试模式下的接口方式

此函数必须在 LoadFile 函数前先调用。避免 OP 自动开启测试。

Example

```
SetInterFaceMode(1);
```

5.12 DisAutoShowLotInfo()

```
int DisAutoShowLotInfo();
```

Return Values

0 : 命令已执行

1 : Testui 没有启动

Remarks

设置标志，在载入测试程序时，禁止自动弹出 Lot 信息输入窗口。

Example

```
DisAutoShowLotInfo();
```

5.13 FileUnLoadWithoutConfirm()

```
int FileUnLoadWithoutConfirm();
```

Return Values

- 0 : 卸载命令已执行
- 1 : 测试程序没有调入
- 1 : 自动测试中，不能卸载程序

Remarks

UnLoad 在卸载测试程序时，没有是否要卸载的提示。

Example

```
FileUnLoadWithoutConfirm();
```

5.14 NewLot()

```
int NewLot();
```

Return Values

- 0 : newlot 命令已执行
- 1 : 测试程序没有调入

Remarks

newlot 功能。

自动化测试模式下(调用了 SetAutoLoadMode 函数)，并且是 op 权限调用此命令，其中内容不可修改。其他情况其内容可修改

Example

```
NewLot();
```

5.15 EndCurrentLog()

```
int EndCurrentLog(int stationid);
```

Parameters

stationid

- 0 : 代表 STATION A
- 1 : 代表 STATION B

Return Values

- 0 : 命令成功
- 1: Control 没启动
- 2: TestUI 不存在
- 3 : 测试程序没有调入

Remarks

停止测试，结束当前批次的测试，保存数据，并初始化界面，已调入的测试程序不卸载。

Example

```
EndCurrentLog(0);
```

5.16 DisableNewLotFunction()

```
int DisableNewLotFunction();
```

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

禁止 new lot 功能

Example

```
DisableNewLotFunction ();
```

5.17 DisableUnloadFunction()

```
int DisableUnloadFunction();
```

Return Values

0 : 命令成功

1 : Testui 没有启动

Remarks

禁止 Unload 功能

Example

```
DisableUnloadFunction();
```

5.18 DisableloadFunction()

```
int DisableloadFunction();
```

Return Values

0 : 命令成功

1 : Testui 没有启动

Remarks

禁止 load 功能

Example

```
DisableloadFunction();
```

5.19 DisableEditFunction()

```
int DisableEditFunction();
```

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

禁止 testui 中的 EDIT 功能

Example

```
DisableEditFunction();
```

5.20 DisableTestFunctionStatus()

```
int DisableTestFunctionStatus();
```

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

置 testui 中的单测功能、连续测试功能、New LOT、批量重测、重测功能为无效状态。

此函数在载入测试程序后可调用。

Example

```
DisableTestFunctionStatus();
```

5.21 SetTestuiRemoteMode()

```
int SetTestuiRemoteMode();
```

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

远程完全控制模式，屏蔽载入、卸载、New Lot、编辑功能。

Example

```
SetTestuiRemoteMode();
```

5.22 SetFailStop()

```
int SetFailStop(int curEnable);
```

Parameters

curEnable

- 0 : 不设置失效停止测试功能
- 1 : 设置失效停止测试功能

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动
- 2 : Control 没有启动
- 1 : 设置值错误
- 2 : 没有载入测试程序
- 3 : 测试程序中已设置，本函数设置无效

Remarks

设置失效停止测试功能

Example

```
SetFailStop(1);
```

6 测试状态与信息获取

6.1 IsTestFileLoad()

```
int IsTestFileLoad(int curstationid);
```

Parameters

Stationed

- 0 : 代表 STATION A
- 1 : 代表 STATION B

Return Values

- 1 : Control 没有启动
- 2 : 当前 station 的 testui 没有启动
- 3 : 测试程序没有已载入
- 0 : 测试程序已载入

Remarks

根据 stationid 号查找测试程序是否已载入.

Example

```
int curFileLoad=IsTestFileLoad (0);
```

6.2 IsDataSave()

```
int IsDataSave(int curstationid);
```

Parameters

Stationed

- 0 : 代表 STATION A
- 1 : 代表 STATION B

Return Values

- 1 : Control 没有启动
- 2 : 当前 station 的 testui 没有启动
- 3 : 数据没有保存
- 4 : 测试没有开始, 数据没有保存
- 0 : 数据已保存

Remarks

根据 stationid 号查找是否已保存数据.

Example

```
int cursaved=IsDataSave(0);
```

6.3 GetCurPgsName()

```
int GetCurPgsName(int curstationid; char* CurPgsName);
```

Parameters

Stationed

- 0 : 代表 STATION A
- 1 : 代表 STATION B

CurPgsName

必须先分配空间,大小为 256 个字符

Return Values

- 1 : Control 没有启动
- 2 : 当前 station 的 testui 没有启动
- 0 : testui 没有调入程序
- N : 实际返回的文件名长度

Remarks

根据 stationid 号查找 testui 已调入的测试程序文件名.

Example

```
char curPgsName [MAX_PATH] = {0};  
int curRes = GetCurPgsName(0, curPgsName);
```

6.4 GetDataLogFileName()

```
int GetDataLogFileName(int curstationid; char* CurDataLogFileName);
```

Parameters

Stationed

0 : 代表 STATION A

1 : 代表 STATION B

CurDataLogFileName

必须先分配空间,大小为 256 个字符

Return Values

-1 : Control 没有启动

-2 : 当前 station 的 testui 没有启动

0 : testui 没有存储

N : 实际返回的文件名长度

Remarks

根据 stationid 号查找 testui 当前存储的 Datalog 文件名.

Example

```
char curDataLogFileName [MAX_PATH] = {0};  
int curRes = GetDataLogFileName(0, curDataLogFileName);
```

6.5 GetExportFilename()

```
int GetExportFilename(char* curmdbfilename; unsigned short FileKind; char*  
datafilename);
```

Parameters

Curmdbfilename

MDB 文件名，用于查验对应的转换输出

FileKind

转换输出文件类型 1: Excel 2: CSV 3: STDF 5: TXT 6: TXT

Datafilename

返回的转换输出文件名，必须先分配空间,大小为 256 个字符

Return Values

0 : 有转换输出

-1 : 没有进行转换输出

-2 : 有转换输出，但转换输出的文件不存在

Remarks

根据 mdb 文件名查找是否有对应的转换输出文件名

Example

```
char curMDBFileName[] = "C:\\AccoTest2021\\AccoTEST System\\\\DATALOG\\Lot 15  
458Wafer9875624_2021 - 11 - 10_1940_40.mdb";  
char curExportFileName[MAX_PATH] = {0};  
int curRes = GetExportFilename(curMDBFileName,2,curExportFileName);
```

6.6 GetInfo()

```
int GetInfo(int curstationid; char* InfoLabel; char* CurInfo);
```

Parameters

Curstationid

0 : 代表 STATION A

1 : 代表 STATION B

InfoLabel

信息标签，目前只增加了三个标签：LotID、TesterID、SubLotID

CurInfo

必须先分配空间,大小为 256 个字符

Return Values

- 1 : Control 没有启动
- 2 : 当前 station 的 testui 没有启动
- N : 实际返回的信息长度

Remarks

根据 stationid 号查找 testui 调入测试程序文件后的相应信息.

Example

```
char curLotInfo [MAX_PATH] = {0};  
int curRes = GetInfo(0, "LotID", curLotInfo);
```

6.7 GetInfoWithSTDFID()

```
int GetInfoWithSTDFID(int curstationid;int curSTDFID; char* CurInfo);
```

Parameters

stationid

- 0 : 代表 STATION A
- 1 : 代表 STATION B

curSTDFID

Lot 信息的 STDFID 存储标识。本 STDFID 请参考函数 AppendStdflotMsg 中的 Lot 信息表

CurInfo

必须先分配空间,大小为 256 个字符

Return Values

- 1 : Control 没有启动
- 2 : 当前 station 的 testui 没有启动
- 3 : curSTDFID 为负

N : 实际返回的信息长度

Remarks

根据 stationid 号、STDFID 号查找 testui 调入测试程序文件后的相应 lot 信息.

Example

```
char curLotInfo[MAX_PATH] = {0};  
int curRes = GetInfoWithSTDFID(0,0,curLotInfo);
```

6.8 CurTestUIExist()

```
int CurTestUIExist(int curstationid);
```

Parameters

Curstationid

0 : 代表 STATION A

1 : 代表 STATION B

Return Values

0 : 当前 TESTUI 句柄对应的窗口不存在

非 0: 当前自动化程序对应的窗口的句柄

Remarks

根据 Station ID 号获取对应 TESTUI 句柄对应的窗口是否存在

Example

```
int curRes = CurTestUIExist(0);
```

6.9 GetTestSiteCount()

```
int GetTestSiteCount();
```

Return Values

-1 : TESTUI 没有启动或没有关联

- 2 : TESTUI 没有载入测试程序
- >0 值 : 当前工位数 (包括 disable 的工位)

Remarks

获取测试工位数

Example

```
int curRes = GetTestSiteCount();
```

6.10 GetTestedResult()

```
int GetTestedResult(int curstationid; byte cursite; int* total; int* failcount);
```

Parameters

curstationid

- 0 : 代表 STATION A
- 1 : 代表 STATION B

Cursite

- 0 : 代表所有工位总统计
- >0 : 代表各工位

Return Values

- 1 : testui 没有启动
- 0 : 返回测试统计信息

Remarks

根据 Station ID 号、工位号码获取当前工位测试的统计值

Example

```
int curTotal=0;  
int curFail=0;  
GetTestedResult(0,0, curTotal, curFail);
```

6.11 GetTestuiTestingStatus()

```
int GetTestuiTestingStatus();
```

Return Values

- 1 : testui 没有启动
- 1 : 测试中, 无法返回信息
- 0 : testui 没有测试, 空闲

Remarks

获取 testui 是否正在测试的状态

Example

```
int curTestingStatus = GetTestuiTestingStatus();
```

6.12 GetExportResult()

```
int GetExportResult();
```

Return Values

- 0 : 所有输出都已完成
- 1: 没有启动转换
- 2: Excel 没有转换完成
- 3: Csv 没有转换完成
- 4: StdF 没有转换完成
- 5: Text 没有转换完成
- 6: 用户自定义输出没有完成
- 7: 本次查询没有查询到结果

Remarks

进行一次输出结果状态查询

Example

```
int curExportResult = GetExportResult();
```

6.13 GetExportResultWithOuttime()

```
int GetExportResultWithOuttime(int delaytime);
```

Parameters

Delaytime

查询超时时间。超时，退出输出结果状态查询。时间单位为毫秒。

Return Values

- 0 : 所有输出都已完成
- 1: 没有启动转换
- 2: Excel 没有转换完成
- 3: Csv 没有转换完成
- 4: StdF 没有转换完成
- 5: Text 没有转换完成
- 6: 用户自定义输出没有完成
- 小于-7000: 部分文件转换失败，不再转换退出

Remarks

进行输出结果状态查询。本查询调用 GetExportResult 函数每隔三秒循环查询一次。如果所有输出都已完成，则直接返回。

Example

```
GetExportResultWithOuttime(300000);
```

6.14 GetExportResultWithMdbVerify()

```
int GetExportResultWithMdbVerify(char* curmdbfilename);
```

Parameters

Curmdbfilename

MDB 文件名。

Return Values

- 0 : 所有输出都已完成
- 1: 没有启动转换
- 2: Excel 没有转换完成
- 3: Csv 没有转换完成
- 4: StdF 没有转换完成
- 5: Text 没有转换完成

Remarks

根据 MDB 文件名进行输出结果状态查询。

Example

```
char curMDBFileName[] = "C:\\AccoTest2021\\AccoTEST System\\\\DATALOG\\Lot 15  
458Wafer9875624_2021 - 11 - 10_1940_40.mdb";  
int curExportResult = GetExportResultWithMdbVerify(curMDBFileName);
```

【高级函数接口】

7 Control 高级函数接口

7.1 StartCalibration()

```
int StartCalibration();
```

Return Values

- 0 : 启动校准
- 1 : 没有启动 Control
- 2 : 没有上电，无法校准

Remarks

启动校准

Example

```
StartCalibration();
```

7.2 GetCalStatus()

```
int GetCalStatus();
```

Return Values

- 1: 没有启动 Control
- 2: 校准软件没有启动
- 0: 未开始校准
- 1: 校准中
- 2: 用户中止校准
- 3: 校准合格
- 4: 校准失败
- 5: 联通性检查合格
- 6: 联通性检查失败

7: 联通性检查中途失败

8: 校准检查中途失败

Remarks

获取校准软件状态

Example

```
GetCalStatus();
```

7.3 CloseCalibration()

```
int CloseCalibration();
```

Return Values

0 : 成功关闭校准

-1: 没有启动 Control

-2: 校准中，无法关闭校准

-3: 校准窗口没找到，无法关闭校准

Remarks

关闭校准窗口

Example

```
CloseCalibration();
```

8 测试信息设置高级函数接口

8.1 RunTestuiWithAllOption()

```
int RunTestuiWithAllOption(char* pgsfilename;int Pcolor;int delaytime; char* SaveFileName; unsigned short OnlyShellClose; unsigned short ExportStd);
```

Parameters

Pgsfilename

测试程序 PGS 文件名

Pcolor

参数显示字体颜色

Delaytime

调入程序函数超时退出时间（ms）

SaveFileName

数据保存文件名(可包含存储路径)

OnlyShellClose

Testui 程序关闭属性（如果为真，则 Testui 必须由 shell 调关闭函数关闭）

ExportStd

输出 STDF 文件

Return Values

0 : 测试程序文件被 STATION A 对应的 Testui 调入

1 : 测试程序文件被 STATION B 对应的 Testui 调入

254 : 调入程序超时退出

255 : Testui 没有启动

Remarks

此函数是下面一系列接口函数的组合，主要包括启动 Testui 程序、传入测试程序文件名、设置参数显示字体颜色、调入程序（包括传入超时退出时间）、传入数据保存文件名（创建数据文件）、设置 Testui 程序关闭属性、设置是否输

出 STDF 文件。

此函数由下面的 RunTestui、SetPgsFileName、SetParamTextColor、SetOnlyShellClose、LoadFile、EnableExportStdF、SetSaveFileName 七个函数组合而成。

Example

```
char curPGSFileName[] = "C:\\AccoTest2021\\AccoTEST  
System\\TESTFILE\\ACM_Example\\ ACM_Example.pgs";  
char curMDBFileName[] = "C:\\AccoTest2021\\AccoTEST System\\DATALOG\\Lot 15  
458Wafer9875624_2021 - 11 - 10_1940_40";  
int curRes = RunTestuiWithAllOption(curPGSFileName,365567,3000, curMDBFileName,1,1);
```

8.2 RunTestuiWithAllOption2()

```
int RunTestuiWithAllOption2(char* pgsfilename; int Pcolor; int delaytime; char*  
SaveFileName; char* LaocalPath; unsigned short OnlyShellClose; unsigned short  
ExportStdF);
```

Parameters

pgsfilename

测试程序 PGS 文件名

Pcolor

参数显示字体颜色

Delaytime

调入程序函数超时退出时间 (ms)

SaveFileName

数据保存文件名(可包含存储路径)

LaocalPath

本地存储路径

OnlyShellClose

Testui 程序关闭属性 (如果为真, 则 Testui 必须由 shell 调关闭函数关闭)

ExportStdF

输出 STDF 文件

Return Values

- 0 : 测试程序文件被 STATION A 对应的 Testui 调入
- 1 : 测试程序文件被 STATION B 对应的 Testui 调入
- 254 : 调入程序超时退出
- 255 : Testui 没有启动

Remarks

此函数是下面一系列接口函数的组合，主要包括启动 Testui 程序、传入测试程序文件名、设置参数显示字体颜色、调入程序（包括传入超时退出时间）、传入数据保存文件名（创建数据文件）、设置本地存储路径、设置 Testui 程序关闭属性、设置是否输出 STDF 文件。它与上一个函数的区别是增加了设置本地存储路径功能。

此函数由下面的 RunTestui、SetPgsFileName、SetParamTextColor、SetOnlyShellClose、LoadFile、EnableExportStdF、SetSaveFileName、SetLocalPath 八个函数组合而成。

Example

```
char curPGSFileName[] = "C:\\AccoTest2021\\AccoTEST  
System\\TESTFILE\\ACM_Example\\ ACM_Example.pgs";  
char curMDBFileName[] = "Z:\\AccoTest2021\\AccoTEST System\\DATALOG\\Lot 15  
458Wafer9875624_2021 - 11 - 10_1940_40";  
char curLocalPath[] = "C:\\AccoTest2021\\DATALOG\\";  
int curRes = RunTestuiWithAllOption2 (curPGSFileName,365567,3000, curMDBFileName,  
curLocalPath,1,1);
```

8.3 EnableUpdateLotIDfromProbe()

```
int EnableUpdateLotIDfromProbe(unsigned short curEnable);
```

Parameters

curEnable

- 0: 不更新 1: 更新

Return Values

0 : 命令成功

Remarks

是否用从 PROBE 获取的 LOTID 信息进行更新

此函数必须在 LoadFile 函数前先调用

Example

```
EnableUpdateLotIDfromProbe(1);
```

8.4 SetLDFDelAttr()

```
int SetLDFDelAttr(int DeleteProperty);
```

Parameters

DeleteProperty

0 : 设置属性为不删除

1 : 设置属性为删除

Return Values

0 : 已置标志

Remarks

设置是否在调程序后删除保存 lot 信息的文件属性。

此函数必须在 LoadFile 函数前先调用

Example

```
SetLDFDelAttr(1);
```

8.5 SetStdExt()

```
int SetStdExt(char* curStdExt);
```

Parameters

curStdfExt

STDF 文件的输出后缀

Return Values

0 : 命令成功

-1 : TestUI 已启动

Remarks

设置 STDF 文件的输出后缀

此函数必须在启动 TestUI 前先调用

Example

```
SetStdExt(".stdf");
```

8.6 SetParamTextColor()

```
int SetParamTextColor(int Pcolor);
```

Parameters

Pcolor

颜色值

Return Values

0 : 成功颜色值

1 : Testui 没有启动

Remarks

设置参数显示字体颜色

此函数必须在 LoadFile 函数前先调用

Example

```
SetParamTextColor(456783);
```

8.7 SetTestProgramSourceInfo()

```
int SetTestProgramSourceInfo(char* SourceInfo);
```

Parameters

SourceInfo

测试程序原始路径或来源

Return Values

0 : 成功

1 : Testui 没有启动

Remarks

设置测试程序原始路径或来源。此信息在数据分析软件中可查看

Example

```
char          curPGSSourceInfo[]          =          "Z:\\\\AccoTest2021\\\\AccoTEST  
System\\\\TESTFILE\\\\ACM_Example\\\\ ACM_Example.pgs";  
SetTestProgramSourceInfo(curPGSSourceInfo);
```

8.8 SummaryExportTxtMode()

```
int SummaryExportTxtMode();
```

Return Values

0 : 命令成功

1 : Testui 没有启动

Remarks

自动输出 text 格式的 Summary 文件。生成的 TXT 文件名与数据文件名相近，
数据文件名+'_sum.txt'

Example

```
SummaryExportTxtMode();
```

8.9 SummaryExportSumMode()

```
int SummaryExportSumMode();
```

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

自动输出text格式的Summary文件. 生成的TXT文件名与数据文件名相近, 数据文件名+ '.sum'

Example

```
SummaryExportSumMode();
```

8.10 SetWholeStdRecord()

```
int SetWholeStdRecord(int RecordTypeCode; int RecordSubCode; void* PRecordContent);
```

Parameters

RecordTypeCode

记录类型值;

RecordSubCode

记录类型

目前仅支持 MIR、SDR、WRR、WIR 四种类型。对应的类型见下表。

类型与子类型表

REC_TYP	描述 及子类型
0	STDF 信息 10 文件属性记录(FAR) 20 修订跟踪记录(ATR)

1	每批次数据集合 10 主信息记录 (MIR) 20 主结果记录 (MRR) 30 part 数量记录 (PCR) 40 硬件档记录 (HBR) 50 软件档记录 (SBR) 60 引脚图记录 (PMR) 62 引脚组记录 (PGR) 63 引脚列表记录 (PLR) 70 重测数据记录 (RDR) 80 工位描述记录 (SDR)
2	圆片数据 10 圆片信息 (WIR) 20 圆片结果记录 (WRR) 30 圆片设置记录 (WCR)
5	per part 数据集合 (可以理解为每只器件的数据) 10 part 信息 (PIR) 20 part 结果记录 (PRR)
10	per test 数据集合 30 测试摘要 (TSR)
15	每次测试执行的数据集合 10 参数测试记录 (PTR) 15 多结果参数记录 (MPR) 20 功能测试记录 (FTR)
20	每程序段数据集合 10 开始程序部分记录 20 结束程序部分记录
50	通用数据 10 通用数据记录 (GDR)

	30 数据文本记录（DTR）
180	为 IMAGE 软件保留
181	为 IG900 软件保留

各记录的内容值结构按照 STDF 文档顺序为：

```
struct SdrRecord    //声明结构体类型
```

```
{
    char* HAND_TYP;    // Handler or prober type
    char* HAND_ID;     // Handler or prober ID
    char* CARD_TYP;    // Probe card type
    char* CARD_ID;     // Probe card ID
    char* LOAD_TYP;    // Load board type
    char* LOAD_ID;     // Load board ID
    char* DIB_TYP;     // DIB board type
    char* DIB_ID;      // DIB board ID
    char* CABL_TYP;    // Interface cable type
    char* CABL_ID;     // Interface cable ID
    char* CONT_TYP;    // Handler contactor type
    char* CONT_ID;     // Handler contactor ID
    char* LASR_TYP;    // Laser type
    char* LASR_ID;     // Laser ID
    char* EXTR_TYP;    // 附加设备类型
    char* EXTR_ID;     // 附加设备ID号
};
```

```
struct MirRecord
```

```
{
    char MODE_COD=0;    // 测试模式（如prod, dev）
    char RTST_COD=0;    // 重测代码 Y =重测 N = 没重测 0 - 9 = 重测次数
    char PROT_COD=0;    // Data protection code
    unsigned short BURN_TIM=65535; // Burn_in time
}
```

```

char CMOD_COD=0;           // Command mode code

char* LOT_ID = "";         //Lot Id
char* PART_TYP = "";       //Part type(or Product Id) 即 DEVICENAME
char* NODE_NAM = "";       //Name of node that generated data
char* TSTR_TYP = "";       //Tester type (测试机类型)
char* JOB_NAM = "";        //任务名 (测试程序名)
char* JOB_REV = "";        //测试程序版本号
char* SBLot_ID = "";       //Sublot ID
char* OPER_NAM = "";       //操作员名称或ID号
char* EXEC_TYP = "";       //Tester executive software type (测试软件类型)
char* EXEC_VER = "";       //测试软件版本
char* TEST_COD = "";       //Test phase or step code, 相当于WORK CENTER
char* TST_TEMP = "";       //测试温度
char* USER_TXT = "";      //Generic user text
char* AUX_FILE = "";       //辅助数据文件名
char* PKG_TYP = "";        //Package type
char* FAMILY_ID = "";      //Product family ID (产品系列ID)
char* DATE_COD = "";       //Date code
char* FACIL_ID = "";       //Test facility (测试设备ID)
char* FLOOR_ID = "";       //Test floor ID (测试台ID)
char* PROC_ID = "";        //Fabrication process ID
char* OPER_FRQ = "";       //Operation frequency or step
char* SPEC_NAM = "";       //Test specification name
char* SPEC_VER = "";       //Test specification version number
char* FLOW_ID = "";        //Test flow ID
char* SETUP_ID = "";       //Test setup ID
char* DSGN_REV = "";       //Device design revision
char* ENG_ID = "";         //Engineering lot ID
char* ROM_COD = "";        //ROM code ID

```

```

char* SERL_NUM = "";          //Tester serial number

char* SUPR_NAM = "";          //Supervisor name or ID
};

struct wrrRecord
{
    char* WAFER_ID = "";       // Wafer ID   length byte = 0
    char* FABWF_ID = "";       // Fab wafer ID   length byte = 0
    char* FRAME_ID = "";       // Wafer frame ID   length byte = 0
    char* MASK_ID = "";        // Wafer mask ID   length byte = 0
    char* USR_DESC = "";       // Wafer描述 (user)   length byte = 0
    char* EXC_DESC = "";       // Wafer描述 (exc)   length byte = 0
};

struct wirRecord
{
    char* WAFER_ID = "";       // Wafer ID   length byte = 0
};

```

Return Values

- 0 : 命令成功
- 1: Testui 没有启动

Remarks

通过 STDF 结构设置 STDF 信息值。此信息传递到 TestUI 后，如果 STDF 结构中的信息项在 lot 信息项中存在对应项，且此 STDF 结构的对应信息项有内容值，则更新 lot 信息项中对应的此信息值；如果 STDF 结构中的信息项在 lot 信息项中存在对应项，但此 STDF 结构的对应信息项没有内容值，则不更新 lot 信息项中对应的此信息值；如果 STDF 结构中的信息项在 lot 信息项中不存在对应项，则将此 STDF 结构的信息项的内容存储到 datalog 中。

此函数必须在载入测试程序之后、测试结束前都能调用。多次调用本函数，以最后一次调用的内容为准。所以最后一次调用输入信息应该最全面。

Example

```
MirRecord * curMir;  
curMir->PART_TYP="USIS"  
int cueSetRes=SetWholeStdRecord(1,10, curMir);
```

9 测试控制高级函数接口

9.1 SetSiteStatus()

```
int SetSiteStatus(byte* pchSite;int sitecnt)
```

Parameters

pchSite

各工位是否有效值

0 : 无效

1 : 有效

Sitecnt

工位数

Return Values

0 : 成功设置

1 : Testui 没有启动

Remarks

设置各 site 是否有效设置各 site 是否有效并实时起作用

Example

```
byte pchSite2[] = {1, 0, 1, 1};  
SetSiteStatus(pchSite2,4);
```

9.2 UpdateSystemEvent()

```
int UpdateSystemEvent();
```

Return Values

0 : 命令成功

-1 : Control 没有启动

Remarks

在启动自动化 UI 后，更新系统消息函数。此接口用于 SECS/GEM 事件触发。

Example

```
UpdateSystemEvent();
```

9.3 SetMaxAutoRunCount()

```
int SetMaxAutoRunCount(int MaxRunTime);
```

Parameters

MaxRunTime

最大运行次数，值为 0 表示关闭

Return Values

- 1 : testui 没有启动
- 2 : testui 没有载入测试程序
- 3 : 最大运行次数小于 0
- 0 : 成功

Remarks

设置最大运行次数

Example

```
SetMaxAutoRunCount(500);
```

9.4 EnableSBL()

```
int EnableSBL(int curEnable;int ClearoldCount;int SampleCount);
```

Parameters

curEnable

- 0 : 不开启
- 1 : 开启 SBL 功能

ClearoldCount

出现 SBL 停机情况后，重启测试，是否清除已有计数

- 0 : 不清除
- 1 : 清除已有计数

SampleCount

SBL 统计最少采样器件数

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

开启并设置 SBL 功能状态

Example

```
EnableSBL(1, 1, 100);
```

9.5 OnlyEnableSBL()

```
int OnlyEnableSBL(int curEnable);
```

Parameters

curEnable

- 0 : 不开启
- 1 : 开启 SBL 功能

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

只开启屏蔽 SBL 功能

Example

```
OnlyEnableSBL(1);
```

9.6 SetSBLimit()

```
int SetSBLimit(unsigned short Bintype;int sbID;double curLimit):
```

Parameters

Bintype

分档类型

0 : 软分档

1 : 硬分档

sbID

分档号

curLimit

此分档对应的 Limit 值

Return Values

0 : 命令成功

1 : Testui 没有启动

2 : Control 没有启动

-1 : 没找到对应 BIN 号

-2 : Bintype 类型错误

Remarks

设置 SBL 功能内容，如果已有 SBL 内容，则更新 Limit 值；如果没有，则自动增加一 SBL 内容值。

Example

```
SetSBLimit(0,3,60);
```

9.7 SetSBLYieldDiffLimit()

```
int SetSBLYieldDiffLimit(unsigned short Bintype;int sbID; double curLimit);
```

Parameters

Bintype

分档类型

0 : 软分档

1 : 硬分档

sbID

分档号

curLimit

此分档对应良率差的 Limit 值

Return Values

0 : 命令成功

1 : Testui 没有启动

2 : Control 没有启动

-1 : 没找到对应 BIN 号

-2 : Bintype 类型错误

Remarks

设置 SBL 功能良率差卡控值，如果已有 SBL 功能良率差卡控值，则更新 Limit 值；如果没有，则自动增加一 SBL 功能良率差卡控值。

Example

```
SetSBLYieldDiffLimit(0,3,0.1);
```

9.8 SetSBLAffectedSite()

```
int SetSBLAffectedSite(int OnlyTotalSite);
```

Parameters

OnlyTotalSite

0 : 分工位

1 : 总工位

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

设置 SBL 功能卡控工位。

Example

```
SetSBLAffectedSite(1);
```

9.9 SetRecentYield()

```
int SetRecentYield(int curEnable;int SampleCount;double Yield);
```

Parameters

curEnable

- 0 : 不开启
- 1 : 开启最近良率超限停机功能

SampleCount

良率统计最少采样器件数

Yield

最近良率超限停机功能的良率值

Return Values

- 0: 命令成功
- 1: Testui 没有启动
- 2: Control 没有启动

Remarks

开启并设置最近良率超限停机功能

Example

```
SetRecentYield(1,100,50);
```

9.10 SetYieldDifferent()

```
int SetYieldDifferent(int curEnable;int SampleCount;double YieldDiff;int clearCount);
```

Parameters

curEnable

- 0 : 不开启
- 1 : 开启良率差异停机功能

SampleCount

良率统计最少采样器件数

Yield

最近良率差异停机功能的良率差异值

ClearoldCount

出现 SYL 停机情况后，重启测试，是否清除已有计数

- 0 : 不清除
- 1 : 清除已有计数

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动
- 2 : Control 没有启动

Remarks

开启并设置良率差异停机功能

Example

```
SetYieldDifferent(1,100,0.5);
```

9.11 SetCorrelationMode()

```
int SetCorrelationMode(int curEnable);
```

Parameters

curEnable

是否设置为样管比对模式

0 : 否

1 : 是

Return Values

-1 : TESTUI 没有启动或没有关联

0 : 成功

Remarks

设置是否进入样管比对模式，如果是样管比对模式，禁止从 TestUI 启动测试，必须从样管比对软件启动测试

Example

```
SetCorrelationMode(1);
```

9.12 LaunchAutoCorrelation()

```
int LaunchAutoCorrelation(char* testProgramFilename; char* CorrelationDataFileName;int curstation);
```

Parameters

testProgramFilename

测试程序名

CorrelationDataFileName

样管比对数据文件名

Curstation

要比对测试站 0: 测试站 A 1: 测试站 B

Return Values

-1 : Control 没有启动或没有关联

-2 : 没有找到样管比对软件

-3 : 测试程序名为空

-4 : 样管比对数据文件名为空

- 5 : 启动样管比对软件失败
- 0 : 启动样管比对软件成功

Remarks

自动化启动样管比对软件

Example

```
char testProgramFilename[] = "C:\\AccoTest2021\\AccoTEST
System\\TESTFILE\\ACM_Example\\ACM_Example.pgs";
char CorrelationDataFileName[] = "C:\\AccoTest2021\\AccoTEST System\\DATALOG\\Lot
15 458Wafer9875624_2021 - 11 - 10_1940_40.csv";
LaunchAutoCorrelation(testProgramFilename, CorrelationDataFileName, 0)
```

9.13 EnableSPL()

```
int EnableSPL(int curEnable;int ClearoldCount);
```

Parameters

curEnable

- 0 : 不开启
- 1 : 开启 SPL 功能

ClearoldCount

- 出现 SPL 停机情况后，重启测试，是否清除已有计数
- 0 : 不清除
 - 1 : 清除已有计数

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

开启并设置 SPL 功能状态

Example

```
EnableSPL(1,1);
```

9.14 SetSPLApplySampleMode()

```
int SetSPLApplySampleMode(int curSampleMode);
```

Parameters

curSampleMode

- 0 : 卡控所有分工位
- 1 : 仅卡控 SPL 测试数达到最小采样数的分工位

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

在有某工位 SPL 测试数达到最小采样数时,设置 SPL 卡控的工位应用范围。

Example

```
SetSPLApplySampleMode(0);
```

9.15 SetSPLYieldSampleCount()

```
int SetSPLYieldSampleCount(int curSampleCount;int OnlyTotalSite);
```

Parameters

curSampleCount

采样数

OnlyTotalSite

- 0 : 分工位卡控
- 1 : 总工位卡控

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

设置 SPL 功能良率卡控采样数及是否总工位卡控状态

Example

```
SetSPLYieldSampleCount(200,0);
```

9.16 SetSPLYieldDIFFSampleCount()

```
int SetSPLYieldDIFFSampleCount(int curSampleCount);
```

Parameters

curSampleCount

采样数

Return Values

0 : 命令成功

1 : Testui 没有启动

Remarks

设置 SPL 功能良率差卡控采样数

Example

```
SetSPLYieldDIFFSampleCount(200);
```

9.17 SetSPLAverageDiffSampleCount()

```
int SetSPLAverageDiffSampleCount(int curSampleCount);
```

Parameters

curSampleCount

采样数

Return Values

0 : 命令成功

1 : Testui 没有启动

Remarks

设置 SPL 功能参数平均值差卡控的合格器件采样数

Example

```
SetSPLAverageDiffSampleCount(200);
```

9.18 SetSPLAverageRangeSampleCount()

```
int SetSPLAverageRangeSampleCount(int curSampleCount;int OnlyTotalSite);
```

Parameters

curSampleCount

采样数

OnlyTotalSite

0 : 分工位卡控

1 : 总工位卡控

Return Values

0 : 命令成功

1 : Testui 没有启动

Remarks

设置 SPL 功能平均值范围卡控的合格器件采样数及是否总工位卡控状态

Example

```
SetSPLAverageRangeSampleCount(100,0);
```

9.19 SetSPLFluctuationRangeSampleCount()

```
int SetSPLFluctuationRangeSampleCount(int curSampleCount;int OnlyTotalSite);
```

Parameters

curSampleCount

采样数

OnlyTotalSite

- 0 : 分工位卡控
- 1 : 总工位卡控

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

设置 SPL 功能参数波动卡控的合格器件采样数及是否总工位卡控状态

Example

```
SetSPLFluctuationRangeSampleCount(200,0);
```

9.20 SetSPLStackingDetectionSampleCount()

```
int SetSPLStackingDetectionSampleCount(int curSampleCount);
```

Parameters

curSampleCount

采样数

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动

Remarks

设置 SPL 功能参数叠料卡控采样数

Example

```
SetSPLStackingDetectionSampleCount(200);
```

9.21 SetSPLYieldLimit()

```
int SetSPLYieldLimit(char* ParamLabel;int SubUnitIndex;double curLimit);
```

Parameters

ParamLabel

参数名

SubUnitIndex

子单元号

curLimit

此参数良率对应的 Limit 值

Return Values

0 : 命令成功

1 : Testui 没有启动

2 : Control 没有启动

-1 : 没找到对应 ParamID

-2 : SubUnitIndex 错误

Remarks

设置 SPL 参数良率超限停机功能内容，如果已有 SPL 内容，则更新 Limit 值；如果没有，则自动增加一 SPL 内容值。

Example

```
SetSPLYieldLimit("RMS", 0, 60);
```

9.22 SetSPLYieldDiffLimit()

```
int SetSPLYieldDiffLimit(char* ParamLabel;int SubUnitIndex;double curDiffLimit);
```

Parameters

ParamLabel

参数名

SubUnitIndex

子单元号

curDiffLimit

此参数各工位良率差对应的 Limit 值

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动
- 2 : Control 没有启动
- 1 : 没找到对应 ParamID
- 2 : SubUnitIndex 错误

Remarks

设置 SPL 良率差异超限停机功能内容，如果已有 SPL 内容，则更新 curDiffLimit 值；如果没有，则自动增加一 SPL 内容值。

Example

```
SetSPLYieldDiffLimit("RMS", 0, 0.6);
```

9.23 SetSPLAverageDiffLimit()

```
int SetSPLAverageDiffLimit(char* ParamLabel;int SubUnitIndex; double curDiffLimit);
```

Parameters

ParamLabel

参数名

SubUnitIndex

子单元号

curDiffLimit

此参数各工位的平均值的差值对应的 Limit 值

Return Values

- 0 : 命令成功
- 1 : Testui 没有启动
- 2 : Control 没有启动
- 1 : 没找到对应 ParamID

-2 : SubUnitIndex 错误

Remarks

设置 SPL 平均值差异停机功能内容, 如果已有 SPL 内容, 则更新 Limit 值; 如果没有, 则自动增加一 SPL 内容值。

Example

```
SetSPLAverageDiffLimit("RMS", 0, 0.6);
```

9.24 SetSPLAverageRangeLimit()

```
int SetSPLAverageRangeLimit(char* ParamLabel;int SubUnitIndex;double AverageMinLimit;  
double AverageMaxLimit;int LimitFlag);
```

Parameters

ParamLabel

参数名

SubUnitIndex

子单元号

AverageMinLimit

此参数平均值对应的最小值

AverageMaxLimit

此参数平均值对应的最大值

LimitFlag

此参数平均值对应的最小值、最大值是否有效的标志

Bit 0 为 1, 最小 Limit 有效

Bit 1 为 1, 最大 Limit 有效

Return Values

0 : 命令成功

1 : Testui 没有启动

2 : Control 没有启动

-1 : 没找到对应 ParamID

-2 : SubUnitIndex 错误

Remarks

设置 SPL 平均值超过范围停机功能内容，如果已有 SPL 内容，则更新 Limit 值；如果没有，则自动增加一 SPL 内容值。

Example

```
SetSPLAverageRangeLimit("RMS", 0, 0.6, 1.6, 3);
```

9.25 SetSPLFluctuationRangeLimit()

```
int SetSPLFluctuationRangeLimit(char* ParamLabel;int SubUnitIndex;double curFluctuationRangeLimit);
```

Parameters

ParamLabel

参数名

SubUnitIndex

子单元号

curFluctuationRangeLimit

所有测试值与此参数平均值的最大偏差值对应的 Limit 值

Return Values

0 : 命令成功

1 : Testui 没有启动

2 : Control 没有启动

-1: 没找到对应 ParamID

-2: SubUnitIndex 错误

Remarks

设置 SPL 测试值波动超过范围停机功能内容，如果已有 SPL 内容，则更新 Limit 值；如果没有，则自动增加一 SPL 内容值。

Example

```
SetSPLFluctuationRangeLimit("RMS", 0, 0.6);
```

9.26 SetSPLStackingDetectionLimit()

```
int SetSPLStackingDetectionLimit(char* ParamLabel;int SubUnitIndex;double curStackingDetectionLimit);
```

Parameters

ParamLabel

参数名

SubUnitIndex

子单元号

curStackingDetectionLimit

参数叠料卡控 Limit 值

Return Values

0 : 命令成功

1: Testui 没有启动

2: Control 没有启动

-1: 没找到对应 ParamID

-2: SubUnitIndex 错误

Remarks

设置 SPL 测试值叠料停机功能内容, 如果已有 SPL 内容, 则更新 Limit 值; 如果没有, 则自动增加一 SPL 内容值。

Example

```
SetSPLStackingDetectionLimit("RMS", 0, 0.2);
```

9.27 ClearSPLSet()

```
int ClearSPLSet(int ClearSetFlag);
```

Parameters

ClearSetFlag

Bit 0=1, 清除良率超限停机功能

Bit 1=1, 清除良率差超限停机功能

Bit 2=1, 清除各工位平均值差超限停机功能

Bit 3=1, 清除平均值超范围停机功能

Bit 4=1, 清除测试波动（所有测试值与平均值的最大偏差）超限停机功能

Bit 5=1, 清除叠料停机功能

Return Values

0 : 命令成功

1 : Testui 没有启动

2 : Control 没有启动

Remarks

清除 SPL 卡控设置功能。

Example

```
ClearSPLSet(7);
```

9.28 SetContinuallyFailStop()

```
int SetContinuallyFailStop(int curEnable;int FailCount);
```

Parameters

curEnable

是否开启 0: 不开启 1: 开启

FailCount

连续失效次数

Return Values

0 : 命令成功

1 : Testui 没有启动

2 : Control 没有启动

Remarks

开启并设置连续失效停机功能。

Example

```
SetContinuallyFailStop(1,20);
```

9.29 SetContinuallyPassStop()

```
int SetContinuallyPassStop(int curEnable;int PassCount);
```

Parameters

curEnable

是否开启 0: 不开启 1: 开启

PassCount

连续合格次数

Return Values

0 : 命令成功

1 : Testui 没有启动

2 : Control 没有启动

Remarks

开启并设置连续合格停机功能。

Example

```
SetContinuallyPassStop (1,200);
```

10 测试信息获取高级函数接口

10.1 GetSiteStatus()

```
int GetSiteStatus(int siteNo);
```

Parameters

siteNo

工位号，从 0 开始

Return Values

- 1 : TESTUI 没有启动或没有关联
- 2 : TESTUI 没有载入测试程序
- 3 : 没有对应 siteNo 的工位
- 0 : 当前工位无效
- 1 : 当前工位有效

Remarks

获取测试工位有效状态

Example

```
int curSiteStatus=GetSiteStatus(0);
```

10.2 GetSubUnitCount()

```
int GetSubUnitCount(char* cur_param_name);
```

Parameters

cur_param_name

参数名称

Return Values

- 1 : TESTUI 没有启动或没有关联

- 2 : TESTUI 没有载入测试程序
- 3 : 没有找到对应的参数
- >0 值 : 当前参数子单元数

Remarks

根据参数名称获取子单元数

Example

```
int curSubNum = GetSubUnitCount("RMS");
```

10.3 GetTestValue()

```
int GetTestValue(int siteNo;char* cur_param_name;int sub_index;double* back_value);
```

Parameters

siteNo

工位号，从 0 开始

cur_param_name

参数名称

sub_index

子单元序号，从 0 开始

back_value

返回的测试值

Return Values

- 1 : TESTUI 没有启动或没有关联
- 2 : TESTUI 没有载入测试程序
- 3 : 没有找到对应的参数名称
- 4 : 子单元序号超范围
- 5 : 没有测试值或测试值无效
- 0 : 成功获取测试值

Remarks

根据工位、参数名、子单元序号获取测试值

Example

```
double back_value=0;  
int curRes = GetTestValue(0,"RMS",0,back_value);
```

10.4 GetParamCount()

```
int GetParamCount(int curstationid);
```

Parameters

curstationid

0 : 代表 STATION A

1 : 代表 STATION B

Return Values

-1 : testui 没有启动

-4 : 测试程序没有载入

>0 : 返回载入测试程序后的参数数值

Remarks

根据 Station ID 号获取当前 Station 载入测试程序后的参数数值

Example

```
int curParamCount= GetParamCount(0);
```

10.5 GetSubUnitCountWithParamIndex()

```
GetSubUnitCountWithParamIndex(int curstationid;int paramindex);
```

Parameters

curstationid

0 : 代表 STATION A

1 : 代表 STATION B

Paramindex

参数序号，从 0 开始

Return Values

- 1 : testui 没有启动
- 4 : 测试程序没有载入
- 5 : paramindex 输入值错误
- >0 值 : 返回载入测试程序后的参数数值

Remarks

根据 Station ID 号获取当前 Station 载入测试程序后的对应参数的测试子单元数

Example

```
int curSubNum=GetSubUnitCountWithParamIndex(0,0);
```

10.6 GetSiteTestResult()

```
int GetSiteTestResult(int curstationid; unsigned short cursite);
```

Parameters

Curstationid

- 0 : 代表 STATION A
- 1 : 代表 STATION B

cursite

- 1: 代表工位 1

Return Values

- 1 : testui 没有启动
- 4 : 测试程序没有载入
- 5 : cursite 输入值错误
- 6 : 当前工位没有测试
- 0 : 当前工位的测试结果合格

1 : 当前工位的测试结果失效

Remarks

根据 Station ID 号获取当前工位当前器件的测试结果（失效、合格）

Example

```
int curTestResult=GetSiteTestResult(0,0);
```

10.7 GetParamTestResult()

```
int GetParamTestResult(int curstationid,unsigned short cursite,int paramindex,int Subindex);
```

Parameters

Curstationid

0 : 代表 STATION A

1 : 代表 STATION B

Cursite

Site 号，从 1 开始， 1 代表工位一

Paramindex

参数序号，从 0 开始

Subindex

子单元序号，从 0 开始

Return Values

-1 : testui 没有启动

-4 : 测试程序没有载入

-5 : cursite 输入值错误

-6 : paramindex 输入值错误

-7 : Subindex 输入值错误

-8 : 当前工位没有测试

0 : 当前工位当前参数的测试结果合格

1 : 当前工位当前参数的测试结果失效

Remarks

根据参数序号、参数子单元号获取当前 Station 当前工位被测器件的测试结果（失效、合格）

Example

```
int curTestResult=GetParamTestResult(0,1,0,0);
```

10.8 GetBinCount()

```
int GetBinCount(int curstationid);
```

Parameters

Curstationid

0 : 代表 STATION A

1 : 代表 STATION B

Return Values

-1 : Control 没有启动

-2 : 当前 station 的 testui 没有启动

N : 返回分档数目

Remarks

根据 stationid 号获取分档数量.

Example

```
int curBinCount=GetBinCount(0);
```

10.9 GetBinInfo()

```
int GetBinInfo(int curstationid;int curBinIndex;int* SBinID;int* HBinID;char* CurBinDescriptionInfo);
```

Parameters

Curstationid

0 : 代表 STATION A

1 : 代表 STATION B

curBinIndex

当前分 BIN 序号

SBinID

当前分 BIN 序号对应的软分 BIN 号;

HBinID

当前分 BIN 序号对应的硬分 BIN 号;

CurBinDescriptionInfo

当前分 BIN 序号对应的分档描述 (必须先分配空间,大小为 256 个字符)

Return Values

-1 : Control 没有启动

-2 : 当前 station 的 testui 没有启动

-3 : 分 BIN 序号 curBinIndex 超过最大分档范围

N : 实际返回的分档描述信息长度

Remarks

根据 stationid 号以及分 BIN 序号获取当前 BIN 的相应信息.

Example

```
int curSBinID=0;
int curHBinID=0;
char curBinDescriptionInfo[MAX_PATH] = {0};
int curGetRes=GetBinInfo(0,3,curSBinID,curHBinID,curBinDescriptionInfo);
```

11 其它接口函数

11.1 CompactFile()

```
int CompactFile(char* curzipfilename; char* cursourcefilename);
```

Parameters

Curzipfilename

Zip 文件文件名

Cursourcefilename

被压缩的文件名

Return Values

0 : 压缩成功

-1 : 压缩错误

Remarks

压缩数据文件

如果数据文件比较大，建议压缩后再进行 FTP 上传

Example

```
char curzipfilename[] = "C:\\AccoTest2021\\AccoTEST System\\DATALOG\\Lot 15  
458Wafer9875624_2021 - 11 - 10_1940_40.zip";  
char cursourcefilename[] = "C:\\AccoTest2021\\AccoTEST System\\DATALOG\\Lot 15  
458Wafer9875624_2021 - 11 - 10_1940_40.mdb";  
CompactFile(curzipfilename, cursourcefilename);
```